COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures & Algorithm	**SECTION:**	CPE21S1
DATE PERFORMED:	10/1/2026	**DATE SUBMITTED:**	10/1/2026
NAME:	Alvarez, Joshua Caleb P.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 10.1 **Hands-on Activity 10.1 Graphs**

1. PROCEDURE OUTPUT

ILO A: Create C++ code for graph implementation utilizing adjacency matrix and adjacency list

A.1. Create a Graph

Following are the basic operations that we can perform on the graph data structure:
Add a vertex: Adds vertex to the graph.
Add an edge: Adds an edge between the two vertices of a graph.
Display the graph vertices: Display the vertices of a graph.

CODE

```
1   //Alvarez, Joshua Caleb P.
2   #include <iostream>
3
4   class Graph{
5   private:
6       int vertices;
7       int edges[10][10];
8
9   public:
10      // Constructor
11      Graph(int v){
12          vertices = v;
13
14          // initialize all edges to 0
15          for (int i = 0; i < vertices; i++){
16              for (int j = 0; j < vertices; j++){
17                  edges[i][j] = 0;
18              }
19          }
20      }
21
```

```
22      // Add an edge between two vertices
23      void addEdge(int start, int end){
24          edges[start][end] = 1;
25          edges[end][start] = 1;
26      }
27
28      // Display the graph vertices
29      void displayVertices(){
30          std::cout << "Graph vertices: ";
31
32          for (int i = 0; i < vertices; i++)
33          {
34              std::cout << i << " ";
35          }
36
37          std::cout << std::endl;
38      }
39
```

```
40      // Display the edges
41      void displayEdges(){
42          std::cout << "Graph edges:" << std::endl;
43
44          for (int i = 0; i < vertices; i++)
45          {
46              for (int j = i + 1; j < vertices; j++)
47              {
48                  if (edges[i][j] == 1)
49                  {
50                      std::cout << i << " - " << j << std::endl;
51                  }
52              }
53          }
54      }
55  };
```

```
57  int main(){
58      // Create a graph with 5 vertices
59      Graph graph(5);
60
61      // Add edges
62      graph.addEdge(0, 1);
63      graph.addEdge(0, 2);
64      graph.addEdge(1, 3);
65      graph.addEdge(2, 4);
66
67      // Display graph vertices
68      graph.displayVertices();
69
70      // Display graph edges
71      graph.displayEdges();
72
73      return 0;
74  }
```

OUTPUT

```
[Running] cd "c:\Users\RYZEN 5
vscode\HOA10P1\"Main_GraphA1
Graph vertices: 0 1 2 3 4
Graph edges:
0 - 1
0 - 2
1 - 3
2 - 4
```

ANALYSIS

This program creates a graph that has five vertices. The function “addEdge” will connect two vertices, while the function “displayVertices” will print all the vertices in the graph. In addition to this, the program will print the edges that have been added which are the numbers in graph vertices.

A.2. Adjacency Matrix

CODE

```cpp
//Alvarez, Joshua Caleb P.
#include <iostream>

class Graph{
private:
    int matrix[5][5];

public:
    // Set matrix to 0
    Graph(){
        for (int i = 0; i < 5; i++){
            for (int j = 0; j < 5; j++)
            {
                matrix[i][j] = 0;
            }
        }
    }

    // Add undirected edge
    void addEdge(int start, int end){
        matrix[start][end] = 1;
        matrix[end][start] = 1;
    }

    // Add weighted edge
    void addWeightedEdge(int start, int end, int weight){
        matrix[start][end] = weight;
    }

    // Display matrix
    void displayMatrix(){
        std::cout << "    A B C D E" << std::endl;
```

```
30      // Display matrix
31      void displayMatrix(){
32          std::cout << "    A B C D E" << std::endl;
33
34          for (int i = 0; i < 5; i++){
35              std::cout << char('A' + i) << "   ";
36
37              for (int j = 0; j < 5; j++){
38                  std::cout << matrix[i][j] << " ";
39              }
40
41              std::cout << std::endl;
42          }
43      }
44  };
45
```

```
46  int main(){
47      // Undirected graph
48      Graph undirected;
49
50      undirected.addEdge(0, 1); // A - B
51      undirected.addEdge(0, 2); // A - C
52      undirected.addEdge(0, 3); // A - D
53      undirected.addEdge(1, 4); // B - E
54      undirected.addEdge(2, 3); // C - D
55      undirected.addEdge(3, 4); // D - E
56
57      std::cout << "Undirected Graph Adjacency Matrix:" << std::endl;
58      undirected.displayMatrix();
59
60      std::cout << std::endl;
61
62      // Weighted graph
63      Graph weighted;
64
```

```
65      weighted.addWeightedEdge(0, 1, 4); // A -> B
66      weighted.addWeightedEdge(0, 2, 3); // A -> C
67      weighted.addWeightedEdge(0, 4, 1); // A -> E
68      weighted.addWeightedEdge(2, 3, 2); // C -> D
69      weighted.addWeightedEdge(3, 1, 1); // D -> B
70      weighted.addWeightedEdge(3, 4, 1); // D -> E
71      weighted.addWeightedEdge(4, 0, 1); // E -> A
72
73      std::cout << "Weighted Graph Adjacency Matrix:" << std::endl;
74      weighted.displayMatrix();
75
76      return 0;
77  }
```

OUTPUT

```
vscode\HOA10P1\"Main_GraphA2
Undirected Graph Adjacency Matrix:
   A B C D E
A  0 1 1 1 0
B  1 0 0 0 1
C  1 0 0 1 0
D  1 0 1 0 1
E  0 1 0 1 0
```

```
Weighted Graph Adjacency Matrix:
   A B C D E
A  0 4 3 0 1
B  0 0 0 0 0
C  0 0 0 2 0
D  0 1 0 0 1
E  1 0 0 0 0
```

ANALYSIS

In the program, the graph is represented using an adjacency matrix. The first matrix represents the undirected graph, where 1 indicates that the vertices are connected, and 0 indicates that the vertices are not connected. On the other hand, the second matrix represents the weighted graph, where numbers indicate the weights of the corresponding edges. Therefore, using an adjacency matrix is convenient to observe the connection and weights of the graph's edges.

A.3. Adjacency List

CODE

vscode > HOA10P1 > Main_graphA3.cpp > display_AdjList(adjNode *, int)

```cpp
//Alvarez, Joshua Caleb P.
#include <iostream>

// stores adjacency list items
struct adjNode{
    int val, cost;
    adjNode* next;
};

// structure to store edges
struct graphEdge{
    int start_ver, end_ver, weight;
};

class DiaGraph{
    // insert new nodes into adjacency list from given graph
    adjNode* getAdjListNode(int value, int weight, adjNode* head){
        adjNode* newNode = new adjNode;

        newNode->val = value;
        newNode->cost = weight;
        newNode->next = head; // point new node to current head

        return newNode;
    }

    int N; // number of nodes in the graph
public:

    adjNode** head; // adjacency list as array of pointers

    // Constructor
    DiaGraph(graphEdge edges[], int n, int N){
        // allocate new node
        head = new adjNode*[N]();

        this->N = N;

        // initialize head pointer for all vertices
        for (int i = 0; i < N; ++i)
            head[i] = nullptr;
```

```
        // construct directed graph by adding edges to it
        for (unsigned i = 0; i < n; i++){
            int start_ver = edges[i].start_ver;
            int end_ver = edges[i].end_ver;
            int weight = edges[i].weight;

            // insert in the beginning
            adjNode* newNode =
                getAdjListNode(end_ver, weight, head[start_ver]);

            // point head pointer to new node
            head[start_ver] = newNode;
        }
    }
```

```
    // Destructor
    ~DiaGraph(){
        for (int i = 0; i < N; i++){
            adjNode* current = head[i];

            while (current != nullptr){
                adjNode* temp = current;
                current = current->next;
                delete temp;
            }
        }

        delete[] head;
    }
};
```

```
// print all adjacent vertices of given vertex
void display_AdjList(adjNode* ptr, int i){
    while (ptr != nullptr){
        std::cout << "(" << i << ", "
                  << ptr->val << ", "
                  << ptr->cost << ") ";

        ptr = ptr->next;
    }

    std::cout << std::endl;
}
```

```
88  // graph implementation
89  int main(){
90      // graph edges array.
91      graphEdge edges[] = {
92          // (x, y, w) -> edge from x to y with weight w
93          {0, 1, 2},
94          {0, 2, 4},
95          {1, 4, 3},
96          {2, 3, 2},
97          {3, 1, 4},
98          {4, 3, 3}
99      };
100
101     int N = 6; // Number of vertices in the graph
102
103     // calculate number of edges
104     int n = sizeof(edges) / sizeof(edges[0]);
105
```

```
106     // construct graph
107     DiaGraph diagraph(edges, n, N);
108
109     // print adjacency list representation of graph
110     std::cout << "Graph adjacency list "
111               << std::endl
112               << "(start_vertex, end_vertex, weight):"
113               << std::endl;
114
115     for (int i = 0; i < N; i++){
116         // display adjacent vertices of vertex i
117         display_AdjList(diagraph.head[i], i);
118     }
119
120     return 0;
121 }
```

OUTPUT

```
Graph adjacency list
(start_vertex, end_vertex, weight):
(0, 2, 4) (0, 1, 2)
(1, 4, 3)
(2, 3, 2)
(3, 1, 4)
(4, 3, 3)
```

ANALYSIS

This program uses an adjacency list to represent the graph. Each vertex has a list of vertices that it is connected to, as well as the weight of the edge connecting them. The program adds the edges to the adjacency list and displays the adjacent vertices along with the weights of the edges. This representation of the graph is more organized since it displays only the vertices that are connected.

ILO B: Create C++ code for implementing graph traversal algorithms such as Breadth-First and Depth-First Search

B.1. Depth-First Search

CODE

```cpp
//Alvarez, Joshua Caleb P.
#include <iostream>
#include <vector>
#include <set>
#include <map>
#include <stack>

template <typename T>
struct Edge{
    size_t src;
    size_t dest;
    T weight;

    bool operator<(const Edge<T>& e) const{
        return weight < e.weight;
    }

    bool operator>(const Edge<T>& e) cons{
        return weight > e.weight;
    }
};

template <typename T>
class Graph;
```

```cpp
template <typename T>
class Graph;

template <typename T>
std::ostream& operator<<(std::ostream& os, const Graph<T>& G){
    for (size_t i = 1; i <= G.vertices(); i++){
        os << i << ":\t";

        std::vector<Edge<T>> edges = G.outgoing_edges(i);

        for (const auto& e : edges){
            os << "{" << e.dest << ": " << e.weight << "}, ";
        }

        os << std::endl;
    }

    return os;
}
```

```cpp
template <typename T>
class Graph{
public:
    // Create graph
    Graph(size_t N) : V(N){
    }

    // Return vertices
    size_t vertices() const{
        return V;
    }

    // Return edges
    const std::vector<Edge<T>>& edges() const{
        return edge_list;
    }

    // Add an edge
    void add_edge(const Edge<T>& e){
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V){
            edge_list.push_back(e);
        }
    }

    // Get outgoing edges
    std::vector<Edge<T>> outgoing_edges(size_t v) const{
        std::vector<Edge<T>> edges_from_v;

        for (const auto& e : edge_list){
            if (e.src == v){
                edges_from_v.push_back(e);
            }
        }

        return edges_from_v;
    }

private:
    size_t V;
    std::vector<Edge<T>> edge_list;
};

// DFS algorithm
template <typename T>
std::vector<size_t> depth_first_search(
    const Graph<T>& G, size_t start){
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
```

```
    stack.push(start);

    while (!stack.empty()){
        size_t current_vertex = stack.top();
        stack.pop();

        if (visited.find(current_vertex) == visited.end()){
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            std::vector<Edge<T>> edges =
                G.outgoing_edges(current_vertex);

            for (const auto& e : edges){
                if (visited.find(e.dest) == visited.end()){
                    stack.push(e.dest);
                }
            }
        }
    }

    return visit_order;
}
```

```
// Create reference graph
template <typename T>
Graph<T> create_reference_graph(){
    Graph<T> G(9);

    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;

    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};

    for (const auto& i : edges){
        for (const auto& j : i.second){
            G.add_edge(Edge<T>{i.first, j.first, j.second});
        }
    }

    return G;
}
```

```cpp
int main(){
    typedef unsigned T;

    Graph<T> G = create_reference_graph<T>();
    std::cout << G << std::endl;
    std::cout << "DFS Order of vertices:" << std::endl;
    std::vector<size_t> dfs_visit_order =
        depth_first_search(G, 1);

    for (size_t v : dfs_visit_order){
        std::cout << v << std::endl;
    }

    return 0;
}
```

OUTPUT

```
vscode\HOA10P1\"Main_DFS_B1
1:  {2: 0}, {5: 0},
2:  {1: 0}, {5: 0}, {4: 0},
3:  {4: 0}, {7: 0},
4:  {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:  {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:  {4: 0}, {7: 0}, {8: 0},
7:  {3: 0}, {6: 0},
8:  {4: 0}, {5: 0}, {6: 0},
9:
```

```
DFS Order of vertices:
1
5
8
6
7
3
4
2
```

ANALYSIS

This program implements Depth-First Search to traverse the graph. The search starts at vertex 1, and the algorithm moves to the first unvisited adjacent vertex. Using the search, the program marks the visited vertices so that it does not visit the same vertex twice. However, the search ends when a vertex does not have any unvisited adjacent vertices, and the algorithm moves back to find another unvisited vertex.

B.2. Breadth-First Search

CODE

```cpp
#include <iostream>
#include <vector>
#include <set>
#include <queue>

template <typename T>
class Graph;

template <typename T>
struct Edge{
    size_t src;
    size_t dest;
    T weight;

    bool operator<(const Edge<T>& other) const{
        return weight < other.weight;
    }

    bool operator>(const Edge<T>& other) const{
        return weight > other.weight;
    }
};

template <typename T>
std::ostream& operator<<(std::ostream& os, const Graph<T>& G){
    for (size_t i = 1; i <= G.vertices(); ++i){
        os << i << ": ";

        for (auto e : G.outgoing_edges(i)){
            os << "(" << e.dest << ", " << e.weight << ") ";
        }

        os << std::endl;
    }

    return os;
}
```

```cpp
template <typename T>
class Graph{
public:
    Graph(size_t N) : V(N){
    }

    size_t vertices() const{
        return V;
    }

    void add_edge(const Edge<T>& e){
        edge_list.push_back(e);
    }

    std::vector<Edge<T>> outgoing_edges(size_t v) const{
        std::vector<Edge<T>> result;

        for (auto e : edge_list){
            if (e.src == v){
                result.push_back(e);
            }
        }

        return result;
    }

private:
    size_t V;
    std::vector<Edge<T>> edge_list;
};

// Create reference graph
template <typename T>
Graph<T> create_reference_graph(){
    Graph<T> G(9);
```

```cpp
// Create reference graph
template <typename T>
Graph<T> create_reference_graph(){
    Graph<T> G(9);

    std::vector<Edge<T>> edges ={
        {1, 2, 2},{1, 5, 3},{2, 1, 2},
        {2, 5, 5},{2, 4, 1},{3, 4, 2},
        {3, 7, 3},{4, 2, 1},{4, 3, 2},
        {4, 5, 2},{4, 6, 4},{4, 8, 5},
        {5, 1, 3},{5, 2, 5},{5, 4, 2},
        {5, 8, 3},{6, 4, 4},{6, 7, 4},
        {6, 8, 1},{7, 3, 3},{7, 6, 4},
        {8, 4, 5},{8, 5, 3},{8, 6, 1}
    };

    for (auto e : edges){
        G.add_edge(e);
    }
    return G;
}
```

```cpp
// Breadth-First Search
template <typename T>
auto breadth_first_search(const Graph<T>& G, size_t dest){
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    queue.push(1); // BFS always starts from vertex 1

    while (!queue.empty()){
        auto current_vertex = queue.front();
        queue.pop();

        if (visited.find(current_vertex) == visited.end()){
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

            for (auto e : G.outgoing_edges(current_vertex)){
                queue.push(e.dest);
            }
        }
    }

    return visit_order;
}
```

```cpp
// Test BFS
template <typename T>
void test_BFS(){
    auto G = create_reference_graph<T>();

    std::cout << G << std::endl;

    std::cout << "BFS Order of vertices: " << std::endl;

    auto bfs_visit_order = breadth_first_search(G, 1);

    for (auto v : bfs_visit_order){
        std::cout << v << std::endl;
    }
}

int main(){
    using T = unsigned;

    test_BFS<T>();

    return 0;
}
```

OUTPUT

```
vscode\HOA10P1\"Main_BFS_B2
1: (2, 2) (5, 3)
2: (1, 2) (5, 5) (4, 1)
3: (4, 2) (7, 3)
4: (2, 1) (3, 2) (5, 2) (6, 4) (8, 5)
5: (1, 3) (2, 5) (4, 2) (8, 3)
6: (4, 4) (7, 4) (8, 1)
7: (3, 3) (6, 4)
8: (4, 5) (5, 3) (6, 1)
9:
```

```
BFS Order of vertices:
1
2
5
4
8
3
6
7
```

ANALYSIS

This program performs and displays a Depth-First Search (DFS) algorithm to go through the graph. It picks the first unvisited vertex from the initial vertex 1 and goes to the next unvisited vertex. The visited vertices are marked so they can be recognized as such and not visited again. If there are no other unvisited vertices, the search ends and the program looks for other unvisited vertices.

COMPUTER ENGINEERING

QUEZON CITY

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Demonstrate an understanding of graph implementation, operations and traversal methods.

1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from the same vertex. Discuss which algorithm would be most helpful to accomplish this task.

- For me **DFS (Depth-First Search)** is the most convenient algorithm since it allows one to explore one vertex to its maximum, and then backtrack and start exploring the other one. It corresponds to the case when the person has to finish one road and only then goes back and tries other roads.

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.

- The equivalent of DFS in tree traversal is Pre-order traversal since both visit the current node first then explore its children. DFS can be compared with tree traversal using a graph/tree diagram, pseudocode, and code implementation.

3. In the performed code, what data structure is used to implement the Breadth First Search?

- The data structure used to implement **Breadth-First Search (BFS)** is a **Queue**. It follows the **First-In, First-Out (FIFO)** principle, where the vertex added first is processed first.

4. How many times can a node be visited in the BFS?

- A node can be **visited only once** in BFS because the program uses a visited set to mark nodes that have already been visited. This prevents the same node from being processed repeatedly.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Chatgpt , Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance, explanation of code and concepts, grammar correction, and improving the organized written answers**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Ai was used as a guide for understanding the activity, checking grammar, and**

assisting with the code. The final answers, analysis and decisions were reviewed and adjusted by me based on my own understanding of the activity.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.